



Entrega 5 - Saga y Saga Log

GRUPO 16

SERGIO FERNANDO BARRERA MOLANO

202517034

HAROLD ANDRÉS BARTOLO MOSCOSO

202513889

JUAN JOSÉ RESTREPO BONILLA

202516633



La saga: transacción larga "aceptar cotización"

1. Qué transacción se orquesta

La capacidad de negocio **aceptar una cotización** no es una operación de un solo servicio: para que HdA pueda prometerle al cliente "su trabajo queda agendado con el pago garantizado" hacen falta tres contextos acotados, cada uno con su base de datos:

Paso	Servicio	Comando	Evento de éxito	Evento de fallo	Compensación → evento
1	Cotizaciones	AceptarCotizacion	CotizacionAceptada	(regla SoloEmitidaSePuedeAceptar)	RevertirAceptacion → CotizacionRevertida
2	Pagos	RetenerPago	PagoRetenido	(regla MontoDebeSerPositivo)	RevertirPago → PagoRevertido
3	Trabajos	AgendarTrabajo	TrabajoAgendado	TrabajoRechazado (regla ProveedorDebeTenerDisponibilidad)	.(último paso)

Tabla 1. Pasos, comandos, eventos y compensaciones de la transacción larga.

No hay transacción distribuida: cada paso confirma localmente y publica su hecho; si un paso posterior falla, los anteriores se compensan en orden inverso con acciones de negocio (liberar el escrow, revertir la aceptación), nunca con rollback técnico.

2. Decisión: orquestación (y por qué no coreografía)

Se implementó la saga como **orquestación**: un servicio **saga** (contexto acotado *Transacciones*, puerto 5005) decide qué comando sigue, espera el evento de respuesta y registra todo en un **Saga Log** propio.

Razones de negocio:

1. **Visibilidad operativa.** Con 30+ partners B2B2C y SLAs por contrato, soporte necesita responder en una consulta "¿en qué paso está la cotización X y por qué falló?". Con orquestación el estado de cada transacción vive en un solo lugar (**sagas** + **saga_log**); con coreografía habría que reconstruirlo cruzando los logs de tres servicios.



2. **Compensaciones con orden y dependencias.** Liberar el escrow debe ocurrir antes de revertir la aceptación (el pago referencia la cotización aceptada). Un orquestador expresa ese orden explícitamente (`_compensar_siguiente`); en coreografía cada servicio tendría que saber qué otros pasos ya ocurrieron.
3. **Evolución de la transacción.** Agregar un paso (p. ej. *verificar cobertura del seguro* para la expansión a MX/BR/AR) es añadir una entrada a **DEFINICION** en el orquestador; en coreografía es tocar los consumidores de varios servicios.
4. **Los servicios siguen desacoplados.** El orquestador solo conoce contratos (comandos y eventos), no APIs ni bases de otros servicios; y los servicios no conocen al orquestador, solo reaccionan a comandos de su tópico y publican eventos. Se evita el riesgo clásico de la orquestación (un "dios" con lógica de negocio) porque la saga no valida reglas de dominio: las reglas siguen en cada agregado (`ProveedorDebeTenerDisponibilidad` vive en trabajos).

Tradeoff asumido: el orquestador es un componente más que desplegar y escalar (por eso es un microservicio con la misma estructura, base propia e idempotencia) y un punto donde una caída pausa las transacciones nuevas, no las pierde: los comandos de inicio quedan retenidos en `comandos-saga` y los eventos en sus tópicos hasta que vuelve (misma táctica de E7).

Dónde sí usamos coreografía: notificaciones consume los eventos de los tres servicios y reacciona sola. No participa en la transacción (no tiene compensación), así que no necesita orquestador.

3. El agregado Saga y el Saga Log (DDD)

`servicios/saga/src/saga/modulos/saga/dominio/entidades.py:`

- Saga es una raíz de agregación: `id`, `id_cotizacion`, `estado` (`EN_CURSO` → `COMPLETADA` | `COMPENSANDO` → `COMPENSADA` | `FALLIDA`), `paso_actual`, `pasos_completados`, `datos` (contexto que cada comando necesita) y `motivo_fallo`.
- Su comportamiento: `iniciar()`, `aplicar_evento(tipo, data)` (avanza, termina o empieza a compensar) y `_compensar_siguiente()` (orden inverso).
- Cada transición agrega un evento de dominio `EntradaSagaLog`; la infraestructura los persiste como filas de la tabla `saga_log` en la **misma Unidad de Trabajo** que el estado de la saga y la marca de idempotencia del mensaje. Después del commit se publican los comandos que la saga decidió (outbox simple). Así el log nunca miente: si hay fila, el estado cambió, y viceversa.
- Objetos valor: `EstadoSaga`, `Paso`, `TipoEntrada`, y `DEFINICION` (la tabla declarativa de pasos/comandos/eventos/compensaciones).



Tablas (infraestructura/dto.py, Flask-SQLAlchemy, base saga.db propia):

```
sagas      id · id_cotizacion · estado · paso_actual · pasos_completados(JSON) · datos(JSON) ·
           secuencia · motivo_fallo · fecha_inicio · fecha_fin
saga_log   id · id_saga · secuencia · fecha · tipo · paso · servicio · mensaje · detalle · payload(JSON)
           tipo ∈ INICIO · COMANDO_ENVIADO · EVENTO_RECIBIDO · PASO_OK · PASO_FALLIDO ·
               COMPENSACION_ENVIADA · COMPENSACION_OK · FIN
eventos_procesados  id_evento  (idempotencia ante re-entregas del broker)
```

4. Cómo se ve una transacción en el Saga Log

Exitosa (proveedor con disponibilidad), 11 entradas:

1 INICIO		saga	IniciarSagaAceptacion
2 COMANDO_ENVIADO	ACEPTAR_COTIZACION	cotizaciones	AceptarCotizacion
3 EVENTO_RECIBIDO	ACEPTAR_COTIZACION	cotizaciones	CotizacionAceptada
4 PASO_OK	ACEPTAR_COTIZACION	cotizaciones	CotizacionAceptada
5 COMANDO_ENVIADO	RETENER_PAGO	pagos	RetenerPago
6 EVENTO_RECIBIDO	RETENER_PAGO	pagos	PagoRetenido
7 PASO_OK	RETENER_PAGO	pagos	PagoRetenido
8 COMANDO_ENVIADO	AGENDAR_TRABAJO	trabajos	AgendarTrabajo
9 EVENTO_RECIBIDO	AGENDAR_TRABAJO	trabajos	TrabajoAgendado
10 PASO_OK	AGENDAR_TRABAJO	trabajos	TrabajoAgendado
11 FIN		saga	COMPLETADA

Con fallo y compensación (proveedor PRV-SIN-CUPO), 17 entradas:

1..8 igual que arriba			
9 EVENTO_RECIBIDO	AGENDAR_TRABAJO	trabajos	TrabajoRechazado
10 PASO_FALLIDO	AGENDAR_TRABAJO	trabajos	TrabajoRechazado
11 COMPENSACION_ENVIADA	RETENER_PAGO	pagos	RevertirPago
12 EVENTO_RECIBIDO	RETENER_PAGO	pagos	PagoRevertido
13 COMPENSACION_OK	RETENER_PAGO	pagos	PagoRevertido
14 COMPENSACION_ENVIADA	ACEPTAR_COTIZACION	cotizaciones	RevertirAceptacion
15 EVENTO_RECIBIDO	ACEPTAR_COTIZACION	cotizaciones	CotizacionRevertida
16 COMPENSACION_OK	ACEPTAR_COTIZACION	cotizaciones	CotizacionRevertida
17 FIN		saga	COMPENSADA

Estado final del negocio: cotización **EMITIDA** (versión 3: la reversión es un evento más en su event store, no un borrado), pago **LIBERADO**, trabajo **RECHAZADO**, saga **COMPENSADA** con el motivo.



5. Monitoreo: API y cliente SQL

Por el BFF (Postman, carpeta 3. Monitoreo de sagas)

Endpoint	Devuelve
GET /bff/sagas	conteo por estado + lista de sagas
GET /bff/sagas/{id_saga o id_cotizacion}	estado, pasos, datos, motivo y log completo
GET /bff/sagas/{ref}/log	solo el log ordenado
GET /bff/sagas/log?n=50	últimas N entradas de todas las sagas
GET /bff/cotizaciones/{id}/estado	vista de negocio + resumen de la saga (fase = COMPENSADA si aplica)

Tabla 2. Endpoints de monitoreo de sagas expuestos por el BFF.

Con sqlite3 (cliente de base de datos)

La imagen del orquestador incluye el CLI. Desde la raíz del repo (Modo A):

```
docker compose exec saga sqlite3 -header -column /app/src/saga/api/saga.db
```

En Modo B: `sqlite3 -header -column servicios/saga/src/saga/api/saga.db`.

Consultas para validar el workflow:



```
-- Sagas por estado (¿cuántas transacciones completaron, cuántas compensaron?)
SELECT estado, COUNT(*) AS sagas FROM sagas GROUP BY estado;

-- Estado actual de cada transacción (monitoreo)
SELECT substr(id,1,8) AS saga, substr(id_cotizacion,1,8) AS cotizacion, estado, paso_actual,
       pasos_completados, motivo_fallo, fecha_inicio, fecha_fin
FROM sagas ORDER BY fecha_inicio DESC LIMIT 20;

-- Workflow completo de una saga (reemplazar el id)
SELECT secuencia, tipo, paso, servicio, mensaje, detalle
FROM saga_log WHERE id_saga = '<id_saga>' ORDER BY secuencia;

-- Solo las compensaciones ejecutadas (auditoría de fallos)
SELECT substr(id_saga,1,8) AS saga, secuencia, tipo, paso, servicio, mensaje, fecha
FROM saga_log WHERE tipo IN ('PASO_FALLIDO','COMPENSACION_ENVIADA','COMPENSACION_OK')
ORDER BY id_saga, secuencia;

-- Duración de cada transacción (ms) ,dato para el documento de resultados
SELECT substr(id,1,8) AS saga, estado,
       CAST((julianday(fecha_fin) - julianday(fecha_inicio)) * 86400000 AS INTEGER) AS duracion_ms
FROM sagas WHERE fecha_fin IS NOT NULL ORDER BY fecha_inicio DESC;

-- En qué paso fallan las transacciones (para priorizar mejoras)
SELECT paso, mensaje, COUNT(*) AS fallos FROM saga_log WHERE tipo = 'PASO_FALLIDO' GROUP BY paso, mensaje;

-- Transacciones en curso hace más de 1 minuto (alerta: ¿un servicio caído?)
SELECT substr(id,1,8) AS saga, paso_actual, fecha_inicio FROM sagas
WHERE estado IN ('EN_CURSO','COMPENSANDO') AND fecha_inicio < datetime('now', '-1 minute');

-- Consistencia: sagas terminadas cuyo log no termina en FIN (nunca debería devolver filas)
SELECT s.id FROM sagas s WHERE s.estado IN ('COMPLETADA','COMPENSADA')
AND NOT EXISTS (SELECT 1 FROM saga_log l WHERE l.id_saga = s.id AND l.tipo = 'FIN');
```

6. Cómo demostrarla

```
bash escenarios/probar_saga.sh           # ejecuta una transacción exitosa y una compensada y
muestra ambos logs
```

o con Postman: carpetas 1. *Transacción larga EXITOSA* y 2. *Transacción larga con FALLO y COMPENSACIÓN* (Runner). En otra terminal, `docker compose logs -f saga pagos trabajos cotizaciones` muestra las líneas [saga] COMANDO ..., [saga] COMPENSACIÓN ..., [trabajos] EVENTO TrabajoRechazado, [pagos] EVENTO PagoRevertido, [cotizaciones] EVENTO CotizacionRevertida.



7. Idempotencia y reanudación

- Cada mensaje que entra al orquestador (comando de inicio o evento) se registra en `eventos_procesados` en la misma transacción; una re-entrega del broker no avanza la saga dos veces (el escenario E7 lo re-inyecta y lo verifica).
- Si el orquestador cae a mitad de una transacción, al volver consume los eventos retenidos en los tópicos y continúa desde `paso_actual`, el estado está en la base, no en memoria.
- Un evento que llega a una saga ya terminada queda registrado como ignorado (`SagaTerminada`).



© - **Derechos Reservados:** la presente obra, y en general todos sus contenidos, se encuentran protegidos por las normas internacionales y nacionales vigentes sobre propiedad Intelectual, por lo tanto su utilización parcial o total, reproducción, comunicación pública, transformación, distribución, alquiler, préstamo público e importación, total o parcial, en todo o en parte, en formato impreso o digital y en cualquier formato conocido o por conocer, se encuentran prohibidos, y solo serán lícitos en la medida en que se cuente con la autorización previa y expresa por escrito de la Universidad de los Andes.

De igual manera, la utilización de la imagen de las personas, docentes o estudiantes, sin su previa autorización está expresamente prohibida. En caso de incumplirse con lo mencionado, se procederá de conformidad con los reglamentos y políticas de la universidad, sin perjuicio de las demás acciones legales aplicables.
